

Malicious Doc Analysis

Malicious PDF Documents

▼ most commonly encountered **risky** PDF keywords are as follows:

- `/OpenAction (/AA)` : This specifies an action to be performed when the document is opened.
- `/Launch` : This keyword specifies an action to launch an external application or open a file.
- `/JavaScript (/JS)` : Specifies a JavaScript action, while `/JS` defines the actual script to be executed.
- `/Names` : This includes the names of files that will likely be referred to by the PDF itself.
- `/EmbeddedFile` : Used to embed files within the PDF.
- `/URI /SubmitForm` : Defines an action to submit form data to a specified URL.
- `/XFA`
- `/URI`
- `/RichMedia`
- `/ObjStm`

PDF Analysis Process

1. I always make sure to check **metadata** such as author, creation date, and software used to create the PDF (`/Author` , `/CreationDate` , `/Producer`) as well the **file size** and compute hashes (`MD5` , `SHA-256`) for verifying integrity.
2. I then examine all objects present in a PDF file and entries or PDF keywords that start with "/".
3. If required, I will can also perform the dynamic analysis in a sandbox environment.

-
- **PDF ID:** `C:\Tools\MalDoc\PDF\Tools\pdfid\pdfid.py`

- **PDF Parser:** `C:\Tools\MalDoc\PDF\Tools\pdf-parser\pdf-parser.py`

```
C:\> python C:\Tools\MalDoc\PDF\Tools\pdfid\pdfid.py C:\Tools
\MalDoc\PDF\Demo\Samples\AgentTesla\invoice-1580727057.pdf -e
```

```
#-----OR-----
C:\> python C:\Tools\MalDoc\PDF\Tools\pdf-parser\pdf-parser.p
y C:\Tools\MalDoc\PDF\Demo\Samples\AgentTesla\invoice-1580727
057.pdf
```

- I can investigate using `-s openaction` at the end of pdfid.py.
- I then can examine object 4 by using command `-o 4`.
- let's this is how I search for it using `pdf-parser` with `-s` or `--search` parameter.

```
C:\> python C:\Tools\MalDoc\PDF\Tools\pdf-parser\pdf-parser.p
y C:\Tools\MalDoc\PDF\Demo\Samples\AgentTesla\invoice-1580727
057.pdf --search=ObjStm
```

- this is how I decode `object 1` to see the objects present in the stream. I can use `-f` or `--filter` in `pdf-parser` to decode the object stream. This parameter passes the stream object through filters (FlateDecode, ASCIIHexDecode, ASCII85Decode, LZWDecode, and RunLengthDecode only).

[illegible]

The start and end of a dictionary are identified by the symbols `<<` and `>>`.

▼ **!?** My goal is to investigate the PDF sample located at "C:\Tools\MalDoc\PDF\Demo\Samples\RedLineStealer\Price_Quote.pdf" and **figure out the URI embedded in the document.**

1. I ran PDF Parser with the `-a` and theres 1 object → 10 1 URI → 9

```
C:\Tools\MalDoc\PDF\Tools\pdf-parser\pdf-parser.py C:\Tools\MalDoc\PDF\Demo\Samples\RedLineStealer\Price_Quote.pdf -a
```

```

C:\Windows\system32>C:\Tools\MalDoc\PDF\Tools\pdf-parser\pdf-parser.py C:\Tools\MalDoc\PDF\Demo\Samples\RedLineStealer\Price_Quote.pdf -a
This program has not been tested with this version of Python (3.12.4)
Should you encounter problems, please use Python version 3.11.1
Comment: 4
XREF: 2
Trailer: 2
StartXref: 2
Indirect object: 17
Indirect objects with a stream: 4, 10, 19, 25, 26, 28
  6: 4, 9, 11, 24, 25, 27
  /Catalog 1: 1
  /ExtGState 2: 7, 8
  /Font 1: 5
  /FontDescriptor 1: 6
  /Metadata 1: 26
  /ObjStm 1: 19
  /Page 1: 3
  /Pages 1: 2
  /XObject 1: 10
  /XRef 1: 28
Unreferenced indirect objects: 19 0 R, 28 0 R
Unreferenced indirect objects without /ObjStm objects: 28 0 R
Search keywords:
  /URI 1: 9

```

Lets run `-o` and dump it using the `-d`

```

python C:\Tools\MalDoc\PDF\Tools\pdf-parser\pdf-parser.py
C:\Tools\MalDoc\PDF\Demo\Samples\RedLineStealer\Price_Quote.pdf -o 10 -d object10.jpeg

```

```

obj 10 0
Type: /XObject
Referencing:
Contains stream

<<
  /Type /XObject
  /Subtype /Image
  /Width 933
  /Height 655
  /ColorSpace /DeviceRGB
  /BitsPerComponent 8
  /Filter /DCTDecode
  /Interpolate true
  /Length 36695
>>

```

- I wasn't getting my desired outcome of what the image should look like, so I am gonna try and rename it:

```
rename object10.jpeg object10.jpg
```

- worry about that later. let's check references:

```
python C:\Tools\MalDoc\PDF\Tools\pdf-parser\pdf-parser.py
C:\Tools\MalDoc\PDF\Demo\Samples\RedLineStealer\Price_Quote.pdf -r 10
```

3. I used command below to get the answer **Price_Quote.bat** (earlier I accidentally found the answer using peepdf.py)

```
python C:\Tools\MalDoc\PDF\Tools\pdf-parser\pdf-parser.py
C:\Tools\MalDoc\PDF\Demo\Samples\RedLineStealer\Price_Quote.pdf -s /URI
```

Office Files

▼ **!?** I ran **olevba.py** with **-a**

" **C:\Tools\MalDoc\Office\Demo\Samples\QuasarRAT\QuasarRAT.docx** " which shows a list of suspicious keywords.

above says to use: **python olevba.py <office-document>**

```
python C:\Tools\MalDoc\Office\Tools\oletools\olevba.py
C:\Tools\MalDoc\Office\Demo\Samples\QuasarRAT\QuasarRAT.do
cx -a
```

Type	Keyword	Description
AutoExec	Document_Open	Runs when the Word or Publisher document is opened
Suspicious	Open	May open a file
Suspicious	Write	May write to a file (if combined with Open)
Suspicious	adodb.stream	May create a text file
Suspicious	SaveToFile	May create a text file
Suspicious	Shell	May run an executable file or a system command
Suspicious	WScript.Shell	May run an executable file or a system command
Suspicious	Call	May call a DLL using Excel 4 Macros (XLM/XLF)
Suspicious	CreateObject	May create an OLE object
Suspicious	Shell.Application	May run an application (if combined with CreateObject)
Suspicious	microsoft.xmlhttp	May download files from the Internet
Suspicious	Chr	May attempt to obfuscate specific strings (use option --deobf to deobfuscate)
Suspicious	Hex Strings	Hex-encoded strings were detected, may be used to obfuscate strings (option --decode to see all)

Answer = **microsoft.xmlhttp**



RDP Login

```
xfreerdp /u:Administrator /p:P3n#31337@LOG /w:1366 /h:768
/v:10.129.232.180
```

Office Document - VBA Macro Analysis

▼ **!?** I used **olemeta.py** to analyze the properties of this document. I found the author of this document is **admin**

```
python C:\Tools\MalDoc\Office\Tools\oletools\olemeta.py  
C:\Tools\MalDoc\Office\Demo\Samples\QuasarRAT\QuasarRAT.docx
```

Properties from the SummaryInformation stream:

Property	Value
codepage	1252
title	
subject	
author	admin
keywords	
template	Normal.dotm
last_saved_by	oplup
revision_number	4
total_edit_time	180
create_time	2023-11-14 22:52:00
last_saved_time	2023-11-14 22:56:00
num_pages	1
num_words	0
num_chars	1
creating_application	Microsoft Office Word
security	0

1. admin was NOT the answer. I think i need to example a difference document, so Im trying the one below: The Answer is **Mohammed Alkuwari**

```
python C:\Tools\MalDoc\Office\Tools\oletools\olemeta.py C:\Tools\MalDoc\Office\Demo\Samples\Havoc\3dfddb91261f5565596e3f014f9c495a.doc
```



```
python C:\Tools\MalDoc\Office\Tools\oletools\oleobj.py  
C:\Tools\MalDoc\Office\Demo\Samples\SnakeKeylogger\P002603  
7.docx
```